event loop. Instead, we're going to rely on convenient higher-level abstractions that will let us focus on the business logic of our applications rather than spending time handling the processing at the thread level.

Scale and cost optimization are two critical reasons for moving to the cloud, so the reactive paradigm perfectly fits cloud native applications. Scaling applications to support a workload increase becomes less demanding. By using resources more efficiently, you can save money on the computational resources offered by a cloud provider. Another reason for moving to the cloud is resilience, and reactive applications also help with that.

One of the essential features of reactive applications is that they provide non-blocking backpressure (also called *control flow*). This means that consumers can control the amount of data they receive, which lowers the risk of producers sending more data than consumers can handle, which can cause a DoS attack, slowing the application, cascading the failure, or even leading to a total crash.

The reactive paradigm is a solution to the problem of blocking I/O operations that require more threads to handle high concurrency and which may lead to slow or entirely unresponsive applications. Sometimes the paradigm is mistaken as a way to increase the speed of an application. Reactive is about improving scalability and resilience, not speed.

With great powers come great troubles, though. Going reactive is an excellent choice when you expect high traffic and concurrency with fewer computational resources or in streaming scenarios. However, you should also be aware of the additional complexity introduced by such a paradigm. Besides requiring a mindset shift to think in an event-driven way, reactive applications are more challenging to debug and troubleshoot because of the asynchronous I/O. Before rushing to rewrite all your applications to make